

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I Monika P(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Monika P

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

$t1 = x + y$

$t2 = t1 * z$

$t3 = t2 / w$

$t4 = t3 - p$

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p char 1

q double 8

r int 4

s float 4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression:

$\text{if } (x > y \parallel p < q)$

Intermediate code:

if $x > y$ goto L1

if $p < q$ goto L1

goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control

Given:

Instruction Target

200: if x==y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

t1 = x + y

t2 = t1 * z

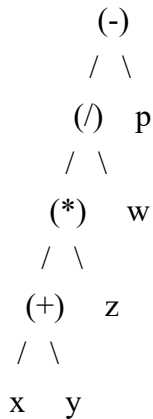
t3 = t2 / w

t4 = t3 - p

Analysis: The sequence evaluates the expression from left to right. First, x and y are added and the result is stored in t1. Next, t1 is multiplied by z and stored in t2. The value of t2 is then divided by w and stored in t3. Finally, p is subtracted from t3 and the final result is stored in t4.

Equivalent expression: $t4 = (((x + y) \times z) / w) - p$

Equivalent expression tree:



Temporary variables improve intermediate code generation by breaking a complex expression into simple operations. Each temporary stores an intermediate result that can be reused by later instructions. This makes the code easier for the compiler to analyze, optimize, and translate into machine instructions. Temporary variables can also reduce repeated computations when the same intermediate result is required more than once. In the given sequence, each temporary is used for the next stage of the calculation, so the computation is organized clearly and efficiently.

Q2. Symbol Table and Memory Allocation

Given symbol table entries:

Variable	Data Type	Size
p	char	1 byte
q	double	8 bytes
r	int	4 bytes
s	float	4 bytes

Memory calculation:

$$\begin{aligned}\text{Total memory} &= 1 + 8 + 4 + 4 \\ &= 17 \text{ bytes}\end{aligned}$$

Therefore, the total memory required for the variables is 17 bytes when padding and alignment are ignored.

The compiler uses the symbol table to keep information about each variable, including its name, data type, and storage size. A char normally requires 1 byte, an int requires 4 bytes, a float requires 4 bytes, and a double requires 8 bytes.

Alignment affects how variables are placed in memory. Some data types must begin at memory addresses that are multiples of a particular boundary. For example, a double is commonly aligned on an 8-byte boundary. To satisfy alignment requirements, the compiler may insert padding bytes between variables. Therefore, although the actual data requires 17 bytes, the physical memory occupied can be greater depending on the order of variables and the target system.

Data types therefore affect both the amount of memory required and the organization of variables in memory. Proper alignment can improve memory-access efficiency, while padding can increase the total memory footprint.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Step-by-step execution:

- Step 1: The compiler calculates $m \times n$ and stores the result in t1.
- Step 2: The compiler calculates $p + q$ and stores the result in t2.
- Step 3: The compiler calculates $m \times n$ again and stores the result in t3.
- Step 4: The compiler subtracts t3 from t2 and stores the result in t4.

Optimization opportunity: The expression $m \times n$ appears twice. Since the values of m and n have not changed between the two occurrences, the second calculation produces exactly the same result as the first. This is a common subexpression and the second multiplication is redundant.

Optimized intermediate code:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1
```

The optimized code eliminates one multiplication operation. This reduces the number of instructions executed and can improve execution speed. It may also reduce register usage and memory traffic. This optimization is called common subexpression elimination, where a previously calculated expression is reused instead of being calculated again.

Q4. Control Flow and Boolean Expression Handling

Given Boolean expression: $\text{if } (x > y \parallel p < q)$

Intermediate code:

```
if x > y goto L1
if p < q goto L1
goto L2
L1: ...
```

Control-flow interpretation: The first condition checked is $x > y$. If $x > y$ is true, the program immediately jumps to L1. Since the OR condition is already true, there is no need to evaluate $p < q$.

If $x > y$ is false, execution proceeds to the second condition, $p < q$. If $p < q$ is true, the program jumps to L1. If $p < q$ is also false, both conditions are false, so the program executes goto L2.

This is an example of short-circuit evaluation. For the OR operator, the second condition is evaluated only when the first condition is false. Therefore, unnecessary condition checking is avoided. This improves execution efficiency and is especially useful when the second condition is expensive to evaluate or depends on the first condition.

Q5. Backpatching and Flow Control

Given instructions:

```
200: if x == y goto _
201: goto _
```

Backpatch lists:

- truelist = {200}
- falselist = {201}

Backpatching is a technique used during intermediate code generation when the destination of a jump is not known at the time the instruction is generated. The compiler temporarily leaves the target address blank and stores the instruction number in a backpatch list.

Here, instruction 200 belongs to the truelist because it is executed when $x == y$ is true. Instruction 201 belongs to the falselist because it is executed when the condition is false.

Suppose the compiler later determines that the true branch must go to instruction 250 and the false branch must go to instruction 260. Backpatching updates the instructions as follows:

```
200: if x == y goto 250
201: goto 260
```

Thus, the blank target in instruction 200 is replaced by 250, and the blank target in instruction 201 is replaced by 260. Backpatching is important for Boolean expression translation, if-else statements, loops, and other control-flow constructs because the final target addresses are often known only after additional intermediate code has been generated.

Conclusion: Backpatching allows the compiler to generate jump instructions before their final destinations are known and then correctly fill in those destinations later. This makes intermediate-code generation flexible and supports complex control-flow structures.